torch.nn.functional.binary_cross_entropy

https://pytorch.org/docs/stable/generated/torch.nn.functional.binary_cross_e

Description:

Compute Binary Cross Entropy between target and input logits. See BCEWithLogitsLoss for details. input (Tensor) – Tensor of arbitrary shape as unnormalized scores (often referred to as logits). target (Tensor) – Tensor of the same shape as input with values between 0 and 1 weight (Tensor, optional) – a manual rescaling weight if provided it's repeated to match input tensor shape

Function Signature

```
torch.nn.functional.binary_cross_entropy_with_logits(input,
target, weight=None, size_average=None, reduce=None,
reduction='mean', pos_weight=None)[source]#

Parameters

Return type
```

Returns:

Tensor

Code Examples

```
>>> input = torch.randn(3, requires_grad=True)
>>> target = torch.empty(3).random_(2)
>>> loss = F.binary_cross_entropy_with_logits(input, target)
>>> loss.backward()
```

Detailed Documentation

Documentation

Compute Binary Cross Entropy between target and input logits.

See BCEWithLogitsLoss for details.

input (Tensor) – Tensor of arbitrary shape as unnormalized scores (often referred to as logits).

target (Tensor) – Tensor of the same shape as input with values between 0 and 1

weight (Tensor, optional) – a manual rescaling weight if provided it’s repeated to match input tensor shape

size_average (bool, optional) – Deprecated (see reduction).

reduce (bool, optional) – Deprecated (see reduction).

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: size_average and reduce are in the process of being deprecated, and in the meantime, specifying either of those two args will override reduction. Default: 'mean'

pos_weight (Tensor, optional) – a weight of positive examples to be broadcasted with target. Must be a tensor with equal size along the class dimension to the number of classes. Pay close attention to PyTorch's broadcasting semantics in order to achieve the desired operations. For a target of size [B, C, H, W] (where B is batch size) pos_weight of size [B, C, H, W] will apply different pos_weights to each element of the batch or [C, H, W] the same pos_weights across the batch. To apply the same positive weight along all spatial dimensions for a 2D multi-class target [C, H, W] use: [C, 1, 1]. Default: None